

INDIAN ARMY CYBER TERRIER QUEST 2026 | PROJECT PROPOSAL

AGNIVEER-SENTINEL

Autonomous Guard Network for Intelligent Vigilance, Electronic Environment Response —
System for Electronic Network Threat Identification & Neutralization, Enhanced Logic

Youthful Agility. Relentless Vigilance. Ultimate AI Defense.

AI-Powered Cyber-Reasoning System (CRS) Prototype for Air-Gapped Military Networks



Introduction, Ideation & Problem Statement

DEFENSE UNDER ZERO-DAY SPECTRUM

- Vulnerable Battlefields: Critical subsystems (battlefield radio comms, radar monitoring, target controllers) operate on native languages (C/C++) vulnerable to memory corruptions.
- Fatal Triage Delays: A manual patch workflow (incident scan → developer triage → code fixing → safety test → deploy) spans days, offering adversaries large windows of exploitation.
- Air-Gapped Realities: Military configurations run on secure air-gapped zones. Cloud-based analysis engines and API dependency lines are strictly prohibited.

THE AGNIVEER-SENTINEL AI KAVACH

- Autonomous CRS Shield: A fully automated local program executing identification, dynamic validation, healing, and verification locally without human operators.
- Double-Gate Security Validation: Utilizes dynamic input fuzzing in sandboxes to verify vulnerabilities, preventing system alerts on false alarms.
- Regression-Guaranteed Auto-Fixes: Leverages a local code reasoner to apply fixes, checking them via a regression suite to guarantee defense lines stay online.

MISSION PROFILE: AUTONOMOUS REPAIR AND PROTECTION SYSTEM OPERATING UNDER 60 SECONDS

System Workflow & Self-Healing Pipeline

01

STATIC SCANNING

Syntactic & AST Analysis

- Scans tactical file structures and binary builds.
- Constructs Abstract Syntax Tree representation.
- Identifies vulnerability signatures and risky library usage (strcpy, system).



02

DYNAMIC FUZZING

Sandbox Crash Testing

- Spawns processes inside secure local sandbox.
- Injects high-velocity boundary inputs (overflow strings, injection payloads).
- Captures process crashes, compiling exploit evidence.



03

AI AUTO-PATCHING

Generative Patch Drafting

- Bundles vulnerable code & crash telemetry logs.
- Prompts the on-premise local code LLM reasoner.
- Generates non-breaking semantic patches.



04

TEST VERIFICATION

Regression Assurance

- Re-compiles codebase with patches applied.
- Runs unit validation suites confirming operational integrity.
- Feeds crash-exploits to verify successful remediation.

↻ CONTINUOUS LOOP: MONITOR → ASSESS → RESOLVE → RE-EVALUATE

Architecture & On-Premise Tech Stack

TECHNOLOGY STACK DETAIL

Terminal HUD interface: Python Rich library for styled layouts, syntax highlighting, and progress tracking.

CLI Orchestration Engine: Python 3.11 with Typer CLI framework for high-speed local processing.

Static AST Analyzers: Custom Python Abstract Syntax Tree scanners & secure coding signature guards.

Dynamic Sandbox Fuzzer: Python Subprocess controller capturing segmentation faults & logic exits.

Local Reasoning LLM: vLLM / Ollama executing quantized Qwen-Coder models 100% offline.

Regression & Build Harness: GNU Make, GCC compilers, and custom verification scripts.

SECURE
REPAIR
VERIFY

TERMINAL COMMAND HUD (Python Rich Library)
ASCII Banners | Live Progress Counters | Colorized Diff Viewer



CRS CORE ENGINE (Typer CLI Orchestrator)
Static AST Scanner | Subprocess Sandbox Fuzzer | Test Harness



LOCAL COGNITIVE INFERENCE (Ollama/vLLM)
On-Premise Qwen-2.5-Coder-7B/14B & Llama-3.1-8B-Instruct

★ TARGET DEPLOYMENT BENCHMARKS (P.O.C.):

- Target 1: C Comms Protocol Server — stack buffer overflow bug.
- Target 2: Python Border Surveillance API — command injection bug.

Expected Deliverables & KPI Metrics

< 60 SECONDS

SELF-HEALING RECOVERY RATE

0% FALSE POSITIVE

VERIFIED VIA DYNAMIC EXPLOITATION

**100% REGRESSION
PASS**

SAFETY-GUARANTEED COMPILATION & TESTS

AUTONOMOUS CRS ENGINE

- Full-featured CLI command orchestrating the CRS loop.
- Subprocess monitoring sandboxes capturing crash vectors.
- On-premise LLM controller interface executing local Qwen-Coder queries.

TACTICAL TERMINAL HUD

- Immersive Terminal Console layouts showing scanning progress.
- Color-coded execution blocks mapping each stage of remediation.
- Live side-by-side terminal syntax diff highlighting patch changes.

AUDIT LOGS & TARGET SUITE

- Autogenerated HTML audit reports summarizing fixes.
- Target folder featuring vulnerable C/Python defense codes.
- Lightweight footprint (<100MB excluding local LLM parameters).